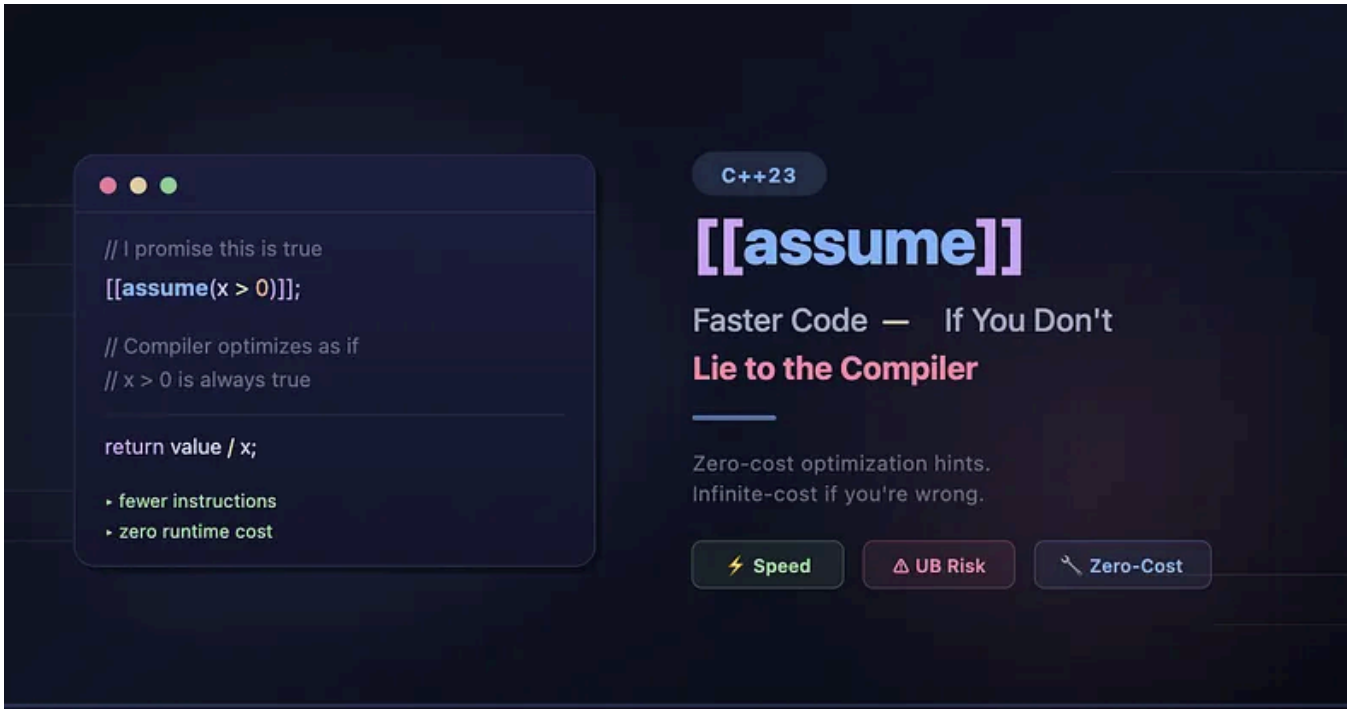
# C++23's [[assume]]: How to Tell Compilers What You Already Know

Zero runtime cost. UB if wrong. The SAFE_ASSUME macro every C++ dev needs.

6 min read · 12 hours ago

Sagar Following

Listen Share More




Every C++ developer wants faster code. We profile, we micro-optimize, we fight for nanoseconds. But sometimes the compiler can't optimize a hot path — not because it's incapable, but because it doesn't know what *you* know about your data.

Is that divisor always positive? Is that vector size always a multiple of 4? Is that pointer always aligned? You know the answer. The compiler doesn't.

C++23 gives you a way to bridge that gap: `[[assume(expr)]]`. It's elegant, it's powerful, and if you misuse it, it will silently destroy your program.

Let's learn how to wield it correctly.

### What Is `[[assume]]` ?

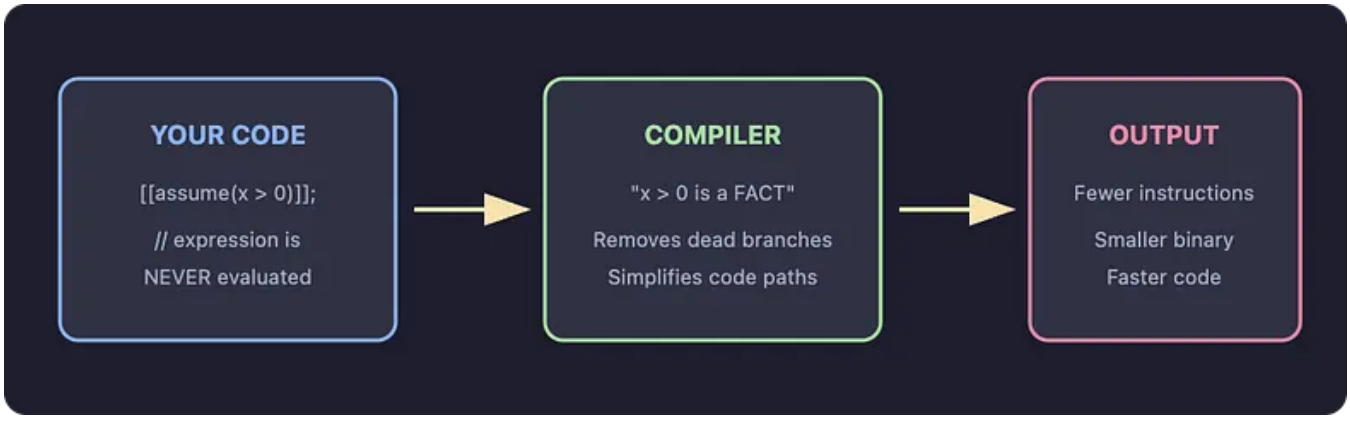
`[[assume(expr)]]` is a C++23 attribute that tells the compiler:

> *"This expression is always `true` at this point. Use that knowledge to generate better code."*

The expression is not executed at runtime, but the compiler assumes it is always true for optimization. It simply trusts you and optimizes accordingly.

```
void process(int x) {
    [[assume(x > 0)]];  // Promise: x is always positive here
    // Compiler now optimizes all downstream code knowing x > 0
}
```

The mental model is this:




### Before C++23: We have to use Compiler-Specific Hacks

Before `[[assume]]`, developers used compiler-specific intrinsics like `__builtin_unreachable()` to hint at impossible conditions:

```
int divide_by_positive(int value, int divisor) {
    if (divisor <= 0) __builtin_unreachable();
    // Compiler infers divisor > 0 because the alternative
    // is "unreachable"
    return value / divisor;
}
```

**With `[[assume]]` (C++23):**

```
int divide_by_positive(int value, int divisor) {
    [[assume(divisor > 0)]];
    return value / divisor;
}
```

**Important distinction:** These two are *not* semantically identical, even though they often produce the same optimized output.

- **`__builtin_unreachable()`** is a **control-flow statement** — it says *"execution never*



*true right now."*

The **`[[assume]]`** form is clearer in intent, portable across compilers, and doesn't require wrapping your assumption in an artificial **`if`** branch. But don't think of it as a drop-in replacement — think of it as the *right tool* for expressing knowledge about your data. Many common cases translate cleanly, but control-flow-heavy patterns involving **`__builtin_unreachable()`** (e.g., inside switch defaults or complex branch structures) may require rethinking rather than simple substitution.

### Here are some practical Use-cases of `[[assume]]` atribute

#### 1. Loop Optimization

```cpp
void process(std::vector<int>& v) {
    [[assume(v.size() % 4 == 0)]];
    // Promise: size is always a multiple of 4

    for (size_t i = 0; i < v.size(); i += 4) {
        // Removes the barrier of generating a scalar remainder
        // loop, enabling SIMD auto-vectorization.
        // Whether the compiler fully vectorizes still depends
        // on target architecture, surrounding code, and
        // optimization level.
        v[i]   *= 2;
        v[i+1] *= 2;
        v[i+2] *= 2;
        v[i+3] *= 2;
    }
}
```

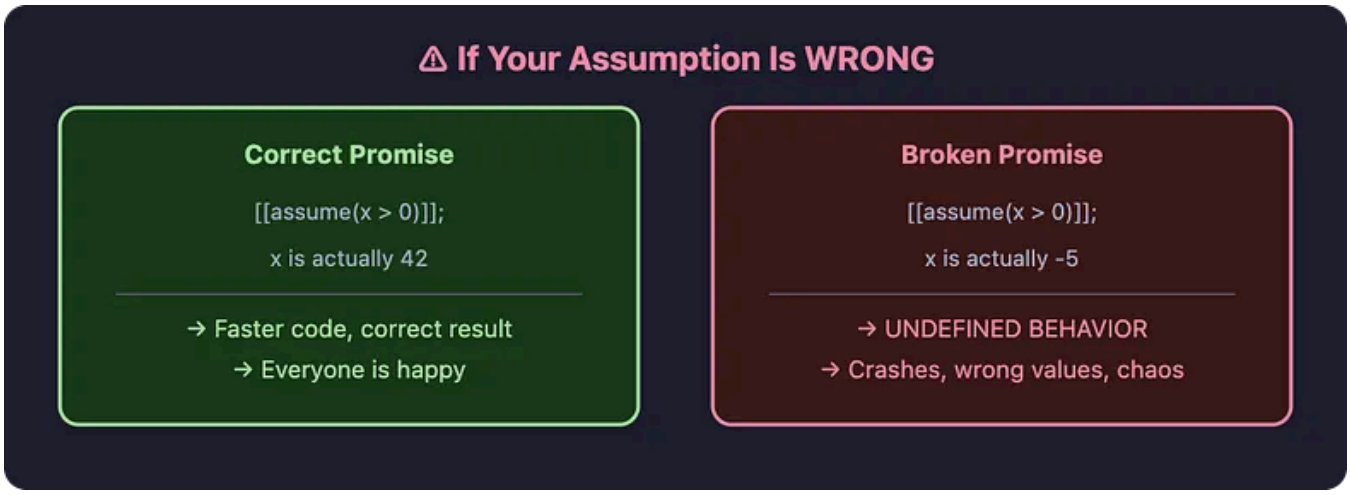
### 2. Pointer Alignment

```cpp
void fast_copy(char* dest, const char* src, size_t n) {
    [[assume(reinterpret_cast<uintptr_t>(dest) % 64 == 0)]];
    [[assume(reinterpret_cast<uintptr_t>(src)  % 64 == 0)]];
    // May allow the compiler to generate aligned
    // load/store instructions instead of unaligned fallbacks
    std::memcpy(dest, src, n);
}
```

### 3. Narrowing Value Ranges

```cpp
int clamp_to_percent(int value) {
    [[assume(value >= 0 && value <= 100)]];
    // Compiler knows the value fits in 7 bits — may enable
    // tighter register usage, elimination of bounds checks
    // in downstream code, or simplified branch logic
    return value;
}
```

### The Danger Zone — UB You Can't See




> ***Key rule:*** *A wrong assumption is* ***undefined behavior*** *— the same category as dereferencing a null pointer. The compiler won't warn you at runtime. There is no safety net.*

But "undefined behavior" can feel abstract. Let's make it concrete.

Consider this function:

```
int f(int x) {
    [[assume(x > 0)]];
    if (x <= 0) return 0;  // Safety check... right?
    return 100 / x;
}
```

A reasonable developer might think: *"Even if the assumption is wrong, my* `if` *guard will catch it."*



**Wrong.** Here's what the compiler sees:

1. `x > 0` is *guaranteed* to be true (you promised).
2. Therefore `x <= 0` is *always false*.

3. Therefore the `if` branch is dead code and can be eliminated.

The compiler optimizes this to:

```
int f(int x) {
    return 100 / x;  // No guard. No check. Nothing.
}
```

Now call `f(0)`. You get a division by zero — not the safe `return 0` you wrote. Call `f(-5)`. You get a negative result from a function that was supposed to only handle positive values.

*Your safety net was removed because your assumption told the compiler it was unnecessary.*

This is the core danger of `[[assume]]`: it doesn't just *enable* optimizations — it gives the compiler permission to **remove code** that contradicts the assumption. Branches, null checks, bounds guards — all of it can vanish silently.

Another Subtle Case:

```
void log_value(int x) {
    [[assume(x != 0)]];
    // ... later, in a different function or inlined context:
    if (x == 0) {
        log("WARNING: x was zero");  // This warning will NEVER fire
    }
    process(1000 / x);
}
```

The diagnostic logging is gone. In production. With no compiler warning. This is why `[[assume]]` demands the same level of rigor as `unsafe` in **Rust** — it's a contract that, if broken, corrupts your program's logic at the optimizer level.

## `[[assume]]` vs. Other Approaches

| APPROACH | STANDARD | RUNTIME COST | UB IF WRONG? |
|---|---|---|---|
| `[[assume(expr)]]` | C++23 | None | Yes |
| `__builtin_unreachable()` | GCC/Clang extension | None | Yes |
| `__assume(expr)` | MSVC extension | None | Yes |
| `assert(expr)` | C/C++ standard | Debug only | No (aborts) |
| `if (!expr) throw` | C++ standard | Always | No |

`[[assume]]` replaces many common uses of compiler-specific hints with a single, portable, standard syntax. It is not a 1:1 replacement for `__builtin_unreachable()` — one expresses *"this point is never reached"* while the other expresses *"this condition holds true"* — but many common optimization patterns map naturally to the `[[assume]]` form.

### Here is the Golden Rule

1. ONLY assume things that are ALWAYS true — not "probably" true.
2. NO side effects in the expression — it's never evaluated!
3. Use assert() during dev, then graduate to [[assume]] in release.
4. Profile first. Only add [[assume]] where it actually helps perf.

### The Assert → Assume Pattern (Recommended)

> *Never assume what you haven't asserted in debug.*

If you can't write an `assert(expr)` that survives your full test suite, you have no business writing `[[assume(expr)]]`. The assumption must be a proven invariant, not a hopeful guess. Every `[[assume]]` in production should have been an `assert()` first.

This is the best-practice pattern used in production codebases:

```
// A macro that validates in debug, optimizes in release
#ifdef NDEBUG
    #define SAFE_ASSUME(expr) [[assume(expr)]]
#else
    #define SAFE_ASSUME(expr) \
        do { assert(expr); [[assume(expr)]]; } while(false)
#endif

double fast_sqrt(double x) {
    SAFE_ASSUME(x >= 0.0);
    // Debug:   crashes if x is negative (assert fires)
    // Release: compiler optimizes assuming x >= 0
    return std::sqrt(x);
}
```

**A note on the macro's debug path:**
The `do { ... } while(false)` wrapper with `[[assume]]` inside a block is well-defined per the C++23 standard — the attribute applies as an attribute-declaration and is valid in any block scope. However, compiler support for `[[assume]]` in all syntactic positions is still maturing. If you encounter issues with a specific compiler version, the fallback is straightforward:

```
#else
    #define SAFE_ASSUME(expr) assert(expr)
#endif
```

This gives you full debug-time validation and gracefully degrades on compilers where attribute placement inside the macro block misbehaves.

#### Compiler Support (as of 2026)

| COMPILER | SUPPORTED SINCE |
| --- | --- |
| GCC | 13+ |
| Clang | 17+ |
| MSVC | 19.37+ (VS 2022 17.7) |

Compile with `-std=c++23` (GCC/Clang) or `/std:c++latest` (MSVC).

#### Final Thoughts

Key takeaways from here:

- **`[[assume(expr)]]`** = a promise to the compiler that `expr` is true
- The expression is never executed — zero runtime cost
- If your promise is wrong → undefined behavior, including silently removed safety checks
- Use it to enable optimizations the compiler can't prove on its own
- Always validate with **`assert()`** first, then promote to **`[[assume]]`**

**`[[assume]]`** is not a tool for writing faster code. It's a tool for telling the compiler what is already true — so it can stop being cautious about things that will never happen.

Use `assert()` to be right. Use `[[assume]]` to be fast. Use both to be a professional.